AI编程的下半场来了？学会用Agent Skill解决编程的痛点问题

腾讯技术工程 2026年3月2日 01:36 广东 164人



作者：bookerzhao

> 如何把 8 年云端经验装进你的 AI 开发工具，让AI从"实习生"变成"持证上岗的专家"。分享如何用 Agent Skill 解决 AI Coding 领域的痛点问题，也分享如何解决 AI 不调用 Skill 等实践技巧。

【推荐语】

最近我们在折腾 Agent Skills，想把云开发（CloudBase）这些年攒下的经验打包给 AI。实战下来发现，最折磨人的不是 AI 不会写代码，而是它写的代码**“只能活在本地”**，以及它**“总是不听规矩”**。

这篇文章主要分享两点实操复盘，希望能帮大家少走点弯路：

1. **让 AI 生成的代码能直接上线**：大家可能都遇到过，AI 写的代码 Demo 感十足，但一上线就全是安全隐患。我们尝试给 AI 注入底座感知，让它学会用底座原生认证代替脆弱的传参，用安全规则代替接口“裸奔”，解决代码“死在本地”的尴尬。
2. **解决“AI 有 Skill 却不爱用”的毛病**：最让人头疼的是，明明配好了 Skills，AI 却视而不见，非要凭直觉盲干。我们会分享如何通过“总纲+插件”的结构，配合简单的工程拦截，把 AI 的技能激活率从 20% 硬拉到 84%。

希望能给同样在研究 Skills 和 AI 开发的朋友一点参考，让 AI 交付的代码不再只是“看着挺美”，而是真正稳健。

现状：Vibe Coding 的 "本地舒适区"

最近，开发者都在享受 **Vibe Coding** 的快感。在用户本地的 `localhost` 一顿操作，UI 漂亮得像成品，但魔法往往在“上线”瞬间戛然而止。



AI 的代码逻辑还不错，但它无法感知真实的后端底座，导致生成的“局部最优解”难以落地，无法简单的生成生产级可用的应用。

世界上最遥远的距离，是从 `localhost` 到真实访问的距离。 AI 填补了代码量的空白，却填补不了工程底座的断层。

有什么办法可以解决这个问题吗？这里就需要提到 Skills。

什么是 Skills

Skills 最早是 Anthropic 在 2025 年 10 月给 Claude Code 加的一个功能，它是一套包含指令、脚本和资源的能力包。**把专业知识、步骤、代码打包成“技能包”**。

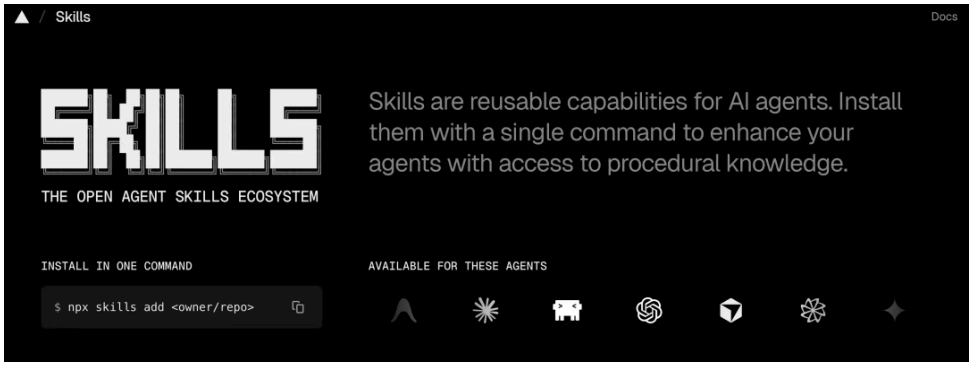


> **Agent Skills 是一种轻量级、开放式的格式，用于通过专业知识和工作流扩展 AI Agent 的能力。**

从本质上讲，一个 Skill 就是一个包含 `SKILL.md` 文件的文件夹。该文件包含元数据（至少包括 `name` 和 `description`）以及告诉 Agent 如何执行特定任务的指令。Skills 还可以捆绑脚本、模板和参考材料。

```
my-skill/
├── SKILL.md          # 核心：指令 + 元数据
├── scripts/          # 可选：可执行代码
├── references/       # 可选：参考文档
└── assets/           # 可选：模板、资源
```

如果把 AI 比作高材生，Skills 就是他的 **“岗位操作手册”。它不改变 AI 的智商，但它通过注入程序性知识**（Procedural Knowledge），让 AI 知道在你的特定环境下，“正确且高效”的操作标准是什么。



Agent Skill [1]在 2025 年 12 月正式成为开放规范，目前已有包括 Claude、Cursor、VS Code、GitHub Copilot、OpenCode等主流AI开发工具宣布兼容支持。

Skills 的工作原理：渐进式加载（Progressive Disclosure）

Skills 通过**渐进式加载**来高效管理上下文（Context），确保 Agent 在拥有强大能力的同时不会因信息过载而变得迟钝：

1. **Discovery（发现）**：启动时，Agent 仅加载每个 Skill 的**名称和描述**。这足以让它在处理请求时，判断哪些 Skill 可能与当前任务相关，而不会耗尽上下文窗口。
2. **Activation（激活）**：当任务与某个 Skill 的描述匹配时，Agent 才会按需将该 Skill 的完整 `SKILL.md` 指令读入当前上下文。
3. **Execution（执行）**：Agent 遵循指令执行任务，并根据需要动态加载引用文件或运行捆绑的脚本代码。

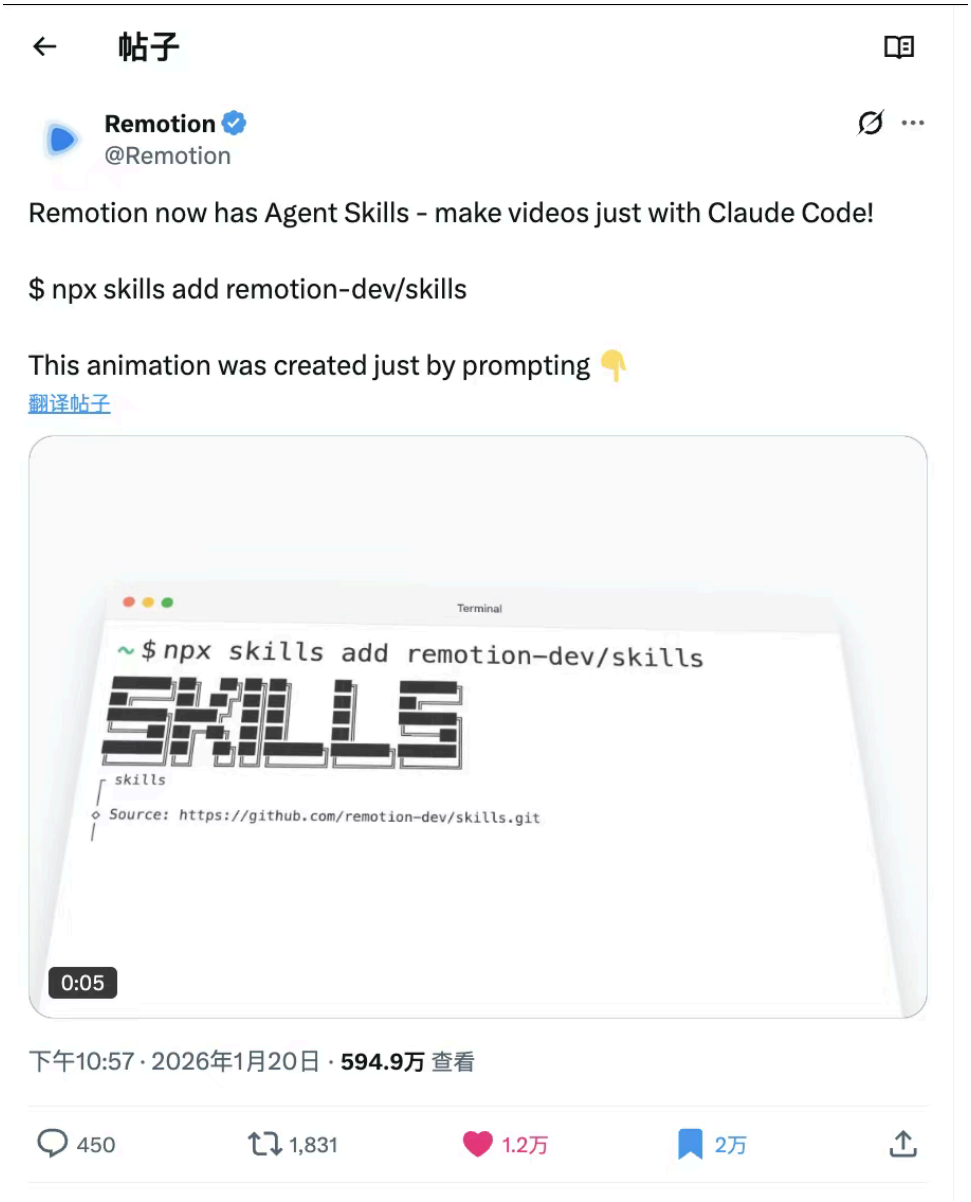
这种方法让 Agent 保持极高的响应速度，同时能够像“随身携带百科全书”一样，在需要时立即获取深度专业知识。

行业大佬们已经在行动：

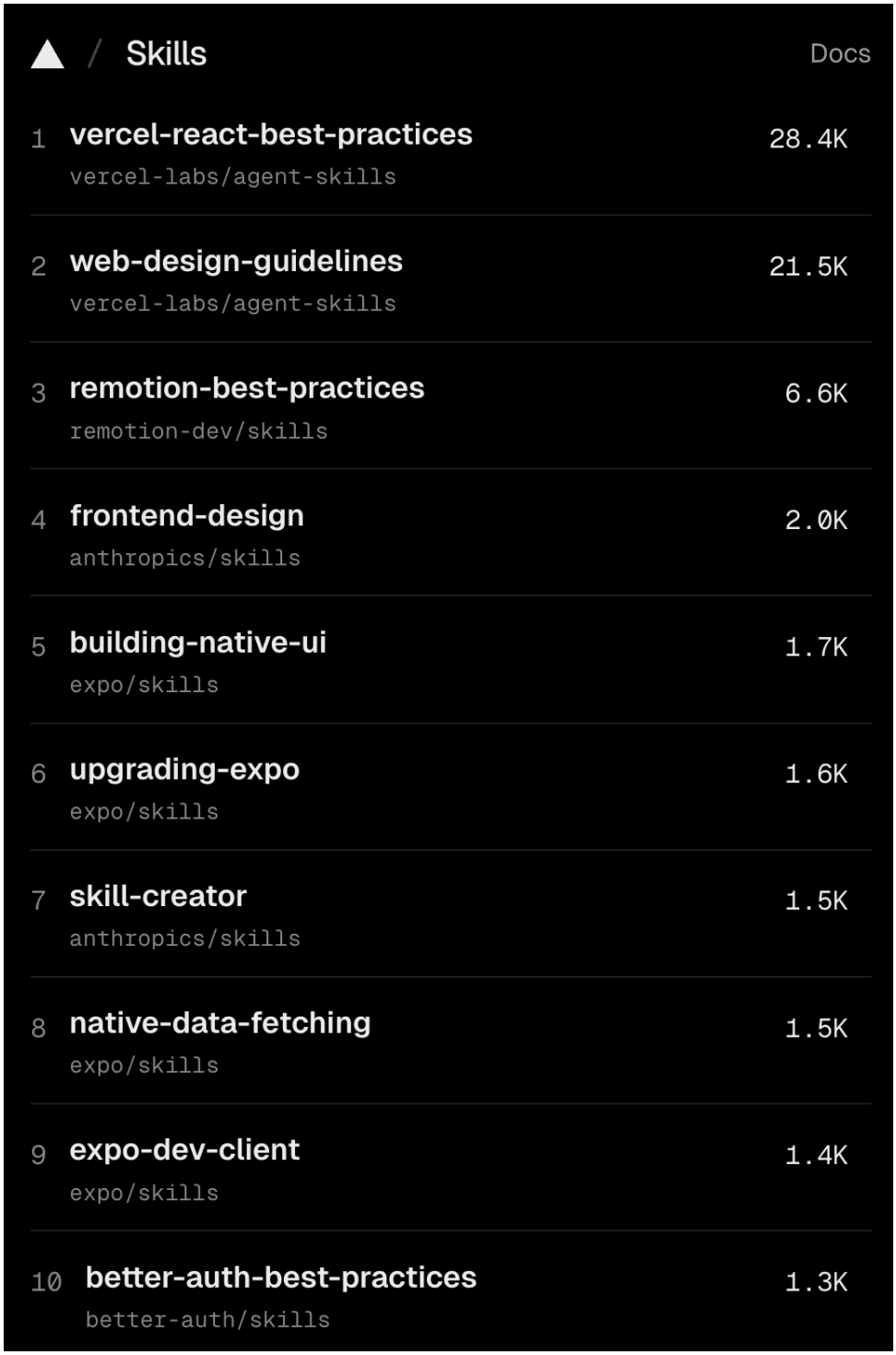
Vercel 发布了 **`react-best-practices`**[2]，解决 AI 乱写 React 导致的性能问题，将 React 专家十余年经验你浓缩为最佳实践。



Remotion 推出**视频制作 Skill**[3]，让 AI 学会用代码"剪辑"视频



Vercel 推出了 skills 命令[4]来快速安装 Skills 到各个工具中，这是当前热门的 skills 列表。



这些 Skills 解决的核心问题是：**让 AI 拥有特定领域的"工程直觉"。**

那么，当你的应用需要落地到云端时，谁来给 AI 注入"后端直觉"？

CloudBase Skills：把 8 年云端经验打包给 AI



云开发 CloudBase[5] 作为自 2018 年起就推出 Serverless 服务的团队，也推出**CloudBase Skills**[6]。

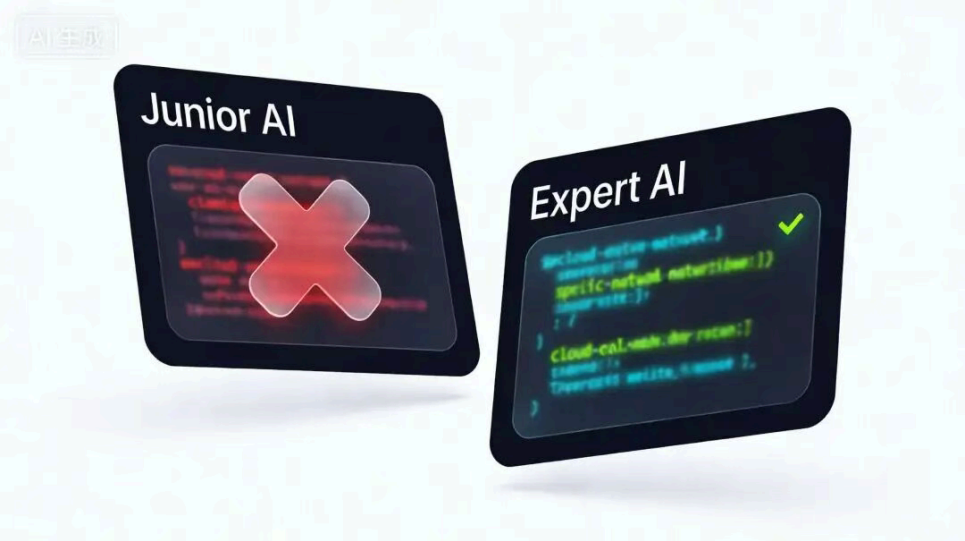
Skills 是软件层面的“岗位手册”，而 CloudBase 则是承载代码落地的“全栈底座”。

许多 AI 生成的代码之所以“死在本地”，是因为 AI 往往只负责写逻辑，却不知道如何对接复杂的生产环境。CloudBase 为 AI 提供了一套高度抽象的基础设施，让 AI 写出的逻辑能无缝运行在公网：

- **全栈托管与部署能力**：支持前端静态网页与后端逻辑的快速部署，让项目从 `localhost` 真正变成可访问的在线 URL。
- **多端原生身份认证**：打通了 Web、小程序等多种身份源。AI 无需手写复杂的登录逻辑，通过 Skills 直接调用底座的认证能力，实现秒级接入。
- **数据库底座**：同时提供文档型（NoSQL）与 SQL 型数据库能力。更重要的是，底座原生集成了面向 C 端的权限控制机制，确保 AI 生成的每一行查询都运行在物理隔离的安全沙箱中，从根源规避越权风险。

它将 CloudBase 支撑**日均 10 亿次 API 调用**、服务**超过 330 万开发者**的真实经验，翻译成了 AI 听得懂的指令。

举一些 CloudBase Skills 如何为 AI 注入生产级标准的场景：



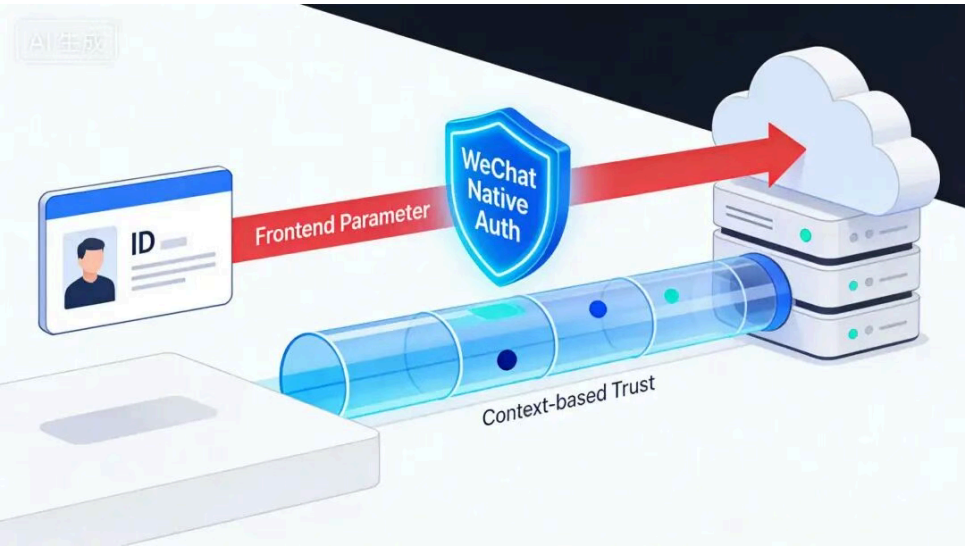
场景一：身份认证——拒绝“相信前端输入”

错误做法

习惯于让前端通过 `userId` 传参给后端。

- **风险**：这是典型的“防君子不防小人”。攻击者只需拦截请求并修改参数，即可实现**横向越权**，访问任何人的私密数据。

正确做法



加载 `auth-wechat` Skill 后，AI 会被强制要求放弃前端传参，转而利用云底座的**原生链路**。

- **Before (脆弱逻辑)**:

```
// 实习生 AI：相信用户传来的 ID
const { openid } = event.params;
const user = await db.collection('users').doc(openid).get();
```

- **After (生产级标准)**:

```
// 专家 AI：利用底座原生上下文，不可伪造
const { OPENID } = cloud.getWXContext();
const user = await db.collection('users').doc(OPENID).get();
```

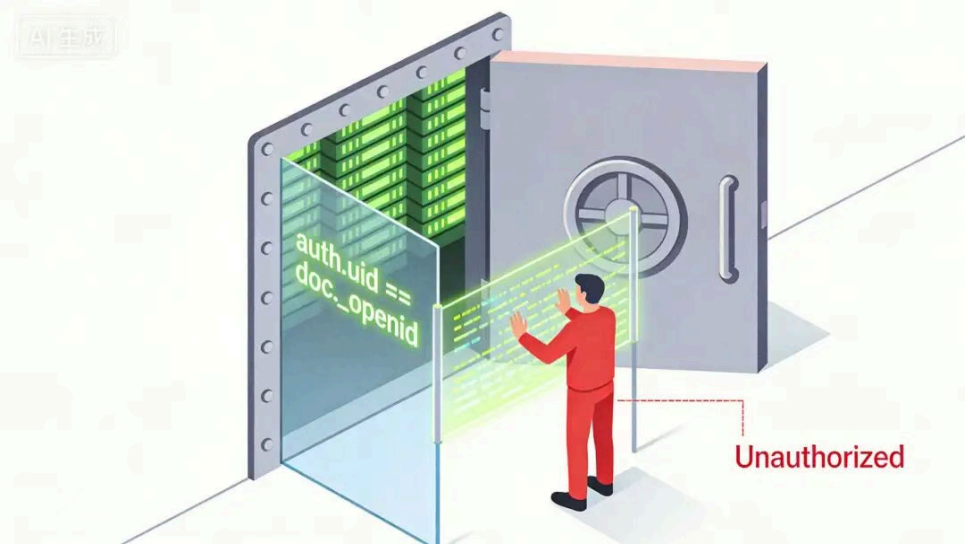
> **工程准则**：安全性应由底座的**原生互信**保证，而非依赖前端输入的自觉。

场景二：数据安全——从“接口裸奔”到“行权限”

错误做法

倾向于直接暴露数据库接口。如果你的 API 逻辑稍有疏忽，数据库对攻击者几乎是“裸奔”状态。

正确做法



Skill 会引导 AI 将权限校验 **下沉到数据库入口**，直接驱动底座的 **安全规则（Security Rules）**。

- **实现改变**：不再单纯写查询逻辑，而是为集合定义 `auth.uid == doc._openid` 规则。
- **价值**：**防御性编程的闭环。** 即使业务逻辑代码出现 Bug，底座依然能从物理层面拦截任何非本人数据的越权修改。

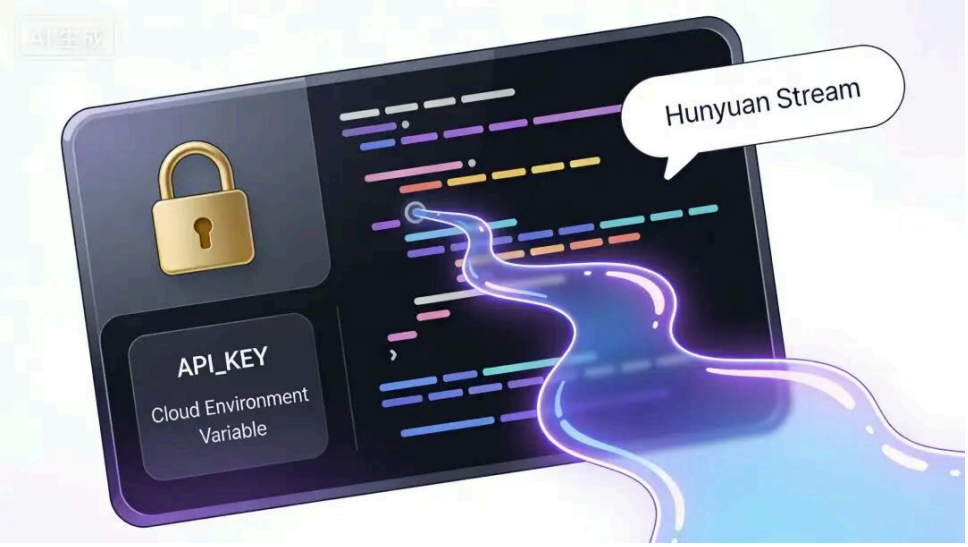
场景三：AI 集成——消灭 Hardcoded，三行代码闭环

接入大模型是当前最热的需求，但 AI 往往给出的是“Demo 级”的代码，完全忽略了生产环境的安全与高可用要求。

错误做法

将 API Key 硬编码在前端，写出一坨混乱的逻辑来处理流式输出（Streaming），既不安全也不稳定。

正确做法



Skill 注入了生产级 AI 接入规范，将复杂的封装逻辑简化为底层 SDK 的原生调用。

- **安全加固**：Key 自动托管在云端环境变量，前端实现 **“零泄露”**。
- **几行代码调用 AI 大模型**，自动处理流式响应。

```
// CloudBase Skills 实现的生产级 AI 调用
const model = wx.cloud.extend.AI.createModel("hunyuan-exp");

const res = await model.streamText({
data: {
    model: "hunyuan-turbos-latest",
    messages: [{ role: "user", content: "介绍一下李白" }],
  }
});

// 使用 textStream 获取增量文本
forawait (const text of res.textStream) {
console.log("文本片段:", text);
}
```

核心价值



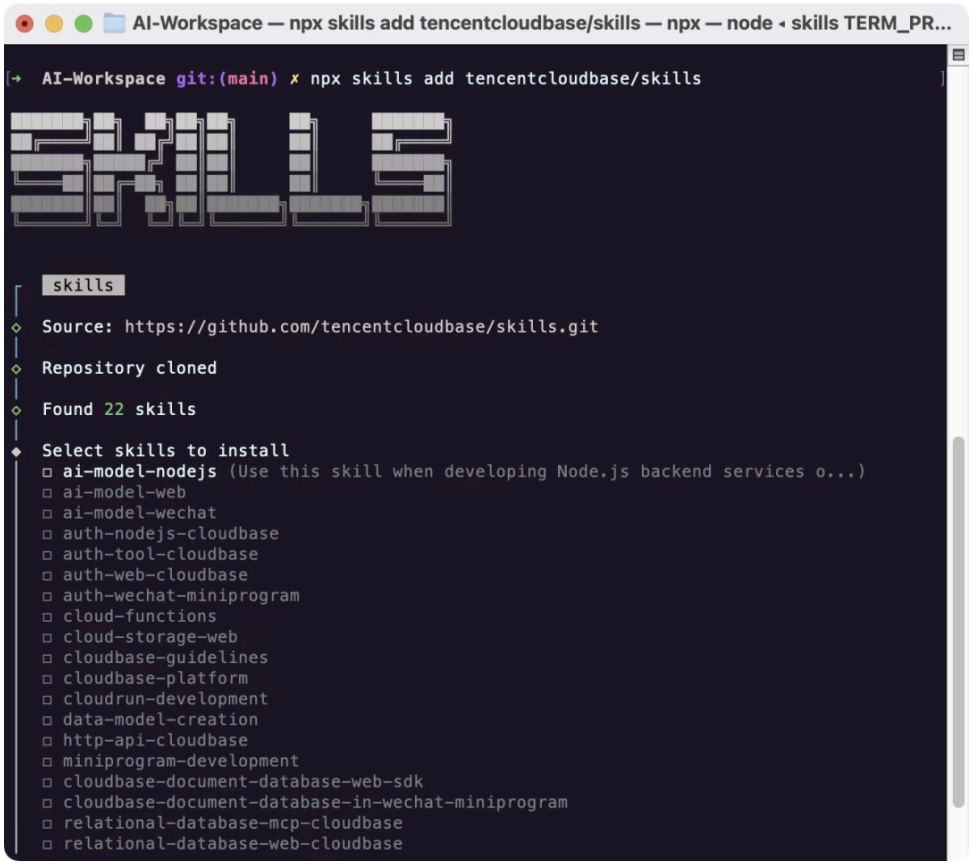
“AI 提供了逻辑的上限，而 CloudBase Skills 守住了工程的下限。”

安装 CloudBase Skills

在你的终端输入以下命令，为你的 AI 助手注入“生产级直觉”：

```
npx skills add tencentcloudbase/skills
```

即可安装我们提供的多个 Skills 到你的开发工具中。



CloudBase Skills 完整列表

我们将 Skills 按照实际开发中的职能进行了归类，确保 AI 在不同环境下调用正确的 SDK 和工具。

这种分类不仅是为了方便开发者查阅，更是为了在 Discovery（发现阶段）降低 AI 的认知负荷，实现精准的按需挂载。

核心理念：环境即边界（Environment as Boundary）

在展开列表前，我们需要阐述这套架构的底层设计：总纲路由，三端隔离。

- **物理隔离**：Web、小程序、Node.js 的同名方法逻辑各异。我们将它们拆分为独立插件，从物理层面杜绝 AI 在小程序里写出 Web SDK 语法的语义污染。
- **总纲引导**：cloudbase-guidelines 是所有任务的默认入口。它像一个语义路由器，先判定项目环境，再指引 AI 激活对应的子 Skill。

完整技能矩阵

分类	核心 Skills	核心职能与实战要点
必读总纲	cloudbase-guidelines	全局入口。判定 Web/小程序/后端环境，负责架构导航与全局防错规矩。
AI 扩展	ai-model-nodejs	服务端专用。支持文本、流式及图像生成。唯一支持 Node SDK 3.16.0+ 图像能力的 Skill。
	ai-model-web / -wechat	前端专用。分别适配浏览器与小程序环境。注意小程序端 API 不同，且不支持图像生成。
身份鉴权	auth-nodejs / -web / -wechat	三端隔离。解决 Web 端的登录、小程序原生身份以及服务端的自定义 Ticket 逻辑。
	auth-tool	MCP 专用。通过 callCloudApi 直接配置短信、匿名、第三方登录等后台开关。
数据库	no-sql-web-sdk / -wx-mp-sdk	文档数据库。适配 Web 与小程序 SDK 的查询、创建、更新、删除及地理位置查询。
	relational-database-tool	关系型数据库。AI 专用 SQL 执行工具，内置生产数据保护机制，严禁裸写 SQL。
	relational-database-web	SQL-on-Web。前端通过 SDK 访问关系型数据库的标准初始化与查询模式。
计算与存储	cloud-functions	Serverless 核心。涵盖运行时选择、部署、日志、异步调用与 HTTP 访问配置。
	cloudrun-development	容器化模式。针对长连接、多语言或 AI Agent 需要的容器云环境开发规范。
	cloud-storage-web	文件管理。Web 环境下的上传、下载、临时链接生成及文件管理最佳实践。
研发流与 UI	spec-workflow	工程标准化。从需求分析到技术设计的标准 SOP，提升复杂项目的确定性。
	ui-design	审美基准。提供生产级的组件设计建议与 UI 规范，确保前端界面具备专业视觉品质。
	web / miniprogram	平台规则。针对 Web 静态托管或小程序开发（含企业微信、wx.cloud 模式）的专项规则。

MCP vs Skills

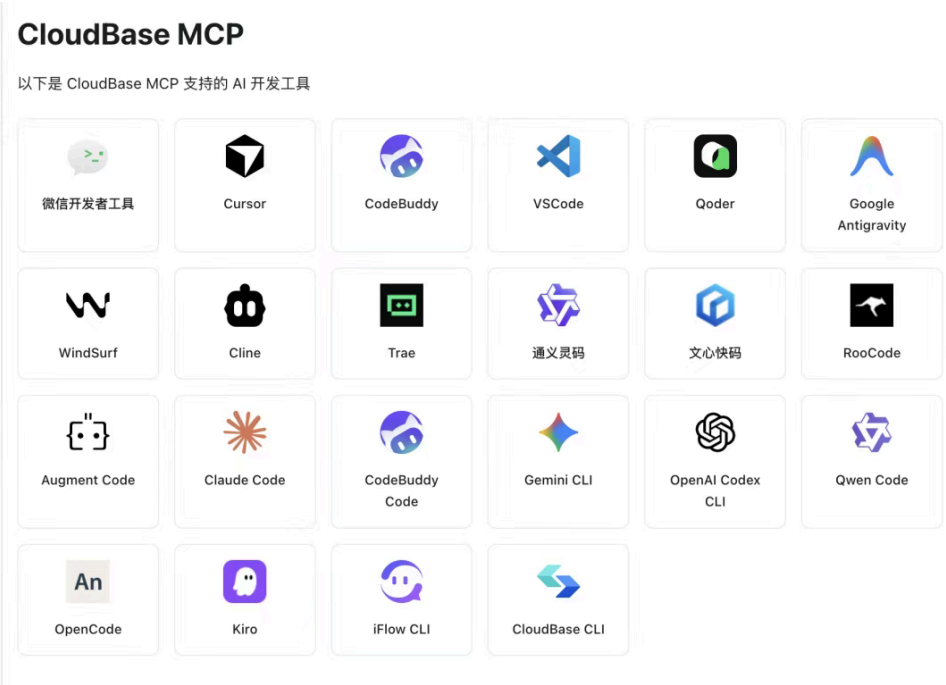
在构建 AI 原生开发生态的道路上，我们始终坚持“先打通，再优化”的逻辑。

CloudBase MCP：早已就绪的“工程双手”

事实上，云开发 CloudBase 在 2025 年上半年的时候就推出了 **CloudBase MCP**[7]。作为 Anthropic 推出的行业标准协议，MCP 解决的是“连接”问题。



通过 MCP，我们让 AI 助手真正拥有了操作腾讯云底座的**结构化权限**。它不再只是在对话框里写代码，而是能直接查询云端状态、创建资源、拉取日志。目前，CloudBase MCP 已经支持了包括 Cursor、Claude Desktop、VS Code 在内的多种主流工具，帮助大量开发者实现了 AI 与云端的物理连接，**查看具体使用指引**[8]。



CloudBase Skills：后发制人的“岗位手册”

既然有了 MCP 这双强有力的“手”，为什么我们还要推出 Skills？

如果说 MCP 是让 AI “有权限”干活，那么 Skills 就是让 AI “懂规矩”干活。

在我们的规划中，Skills 是更上层的能力封装：

- **不仅仅是脚本**：虽然 Skills 规范支持包含脚本（Scripts），但我们现阶段更侧重于**“程序性知识（Procedural Knowledge）”的注入**。
- **更安全、更通用的专家直觉**：正如前文提到的，AI 有了 MCP 虽然能执行脚本，但在云端生产环境，盲目执行脚本不仅不安全，也不符合工程规范。Skills 则是把我们 8 年的云端填坑经验（如行权限隔离、原生上下文认证）变成 AI 的**直觉**。



为什么它们是黄金组合？

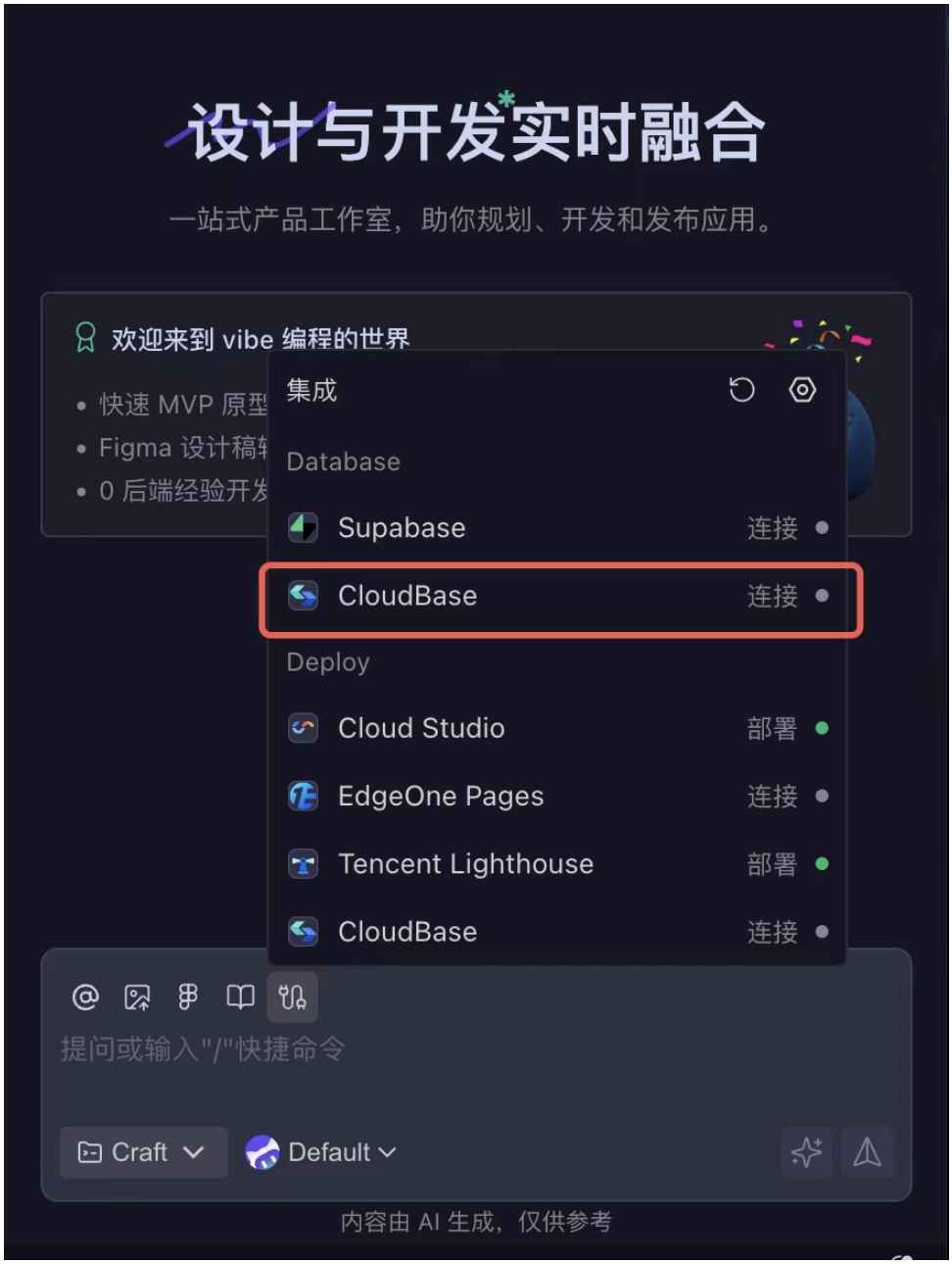
你可以单独使用其中之一，但组合使用才是 AI 编程的“终极形态”：

“MCP 提供了标准化的安全连接，而 Skills 提供了生产级的工程直觉。”

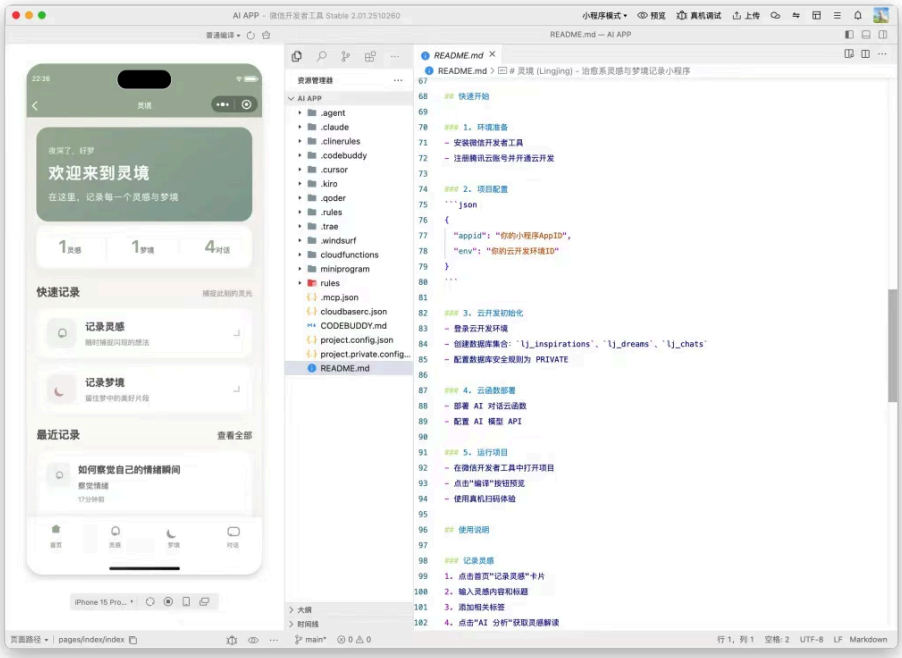
两者搭配，让 AI 从一个 **“力气大但鲁莽的实习生”**，真正进化为一个**“懂规矩、有权限”** 的资深云开发专家。

实战案例

CodeBuddy IDE 中已经内置了 CloudBase MCP 和 Skills，在连接 CloudBase 之后会自动下载 Skills 到你项目中。

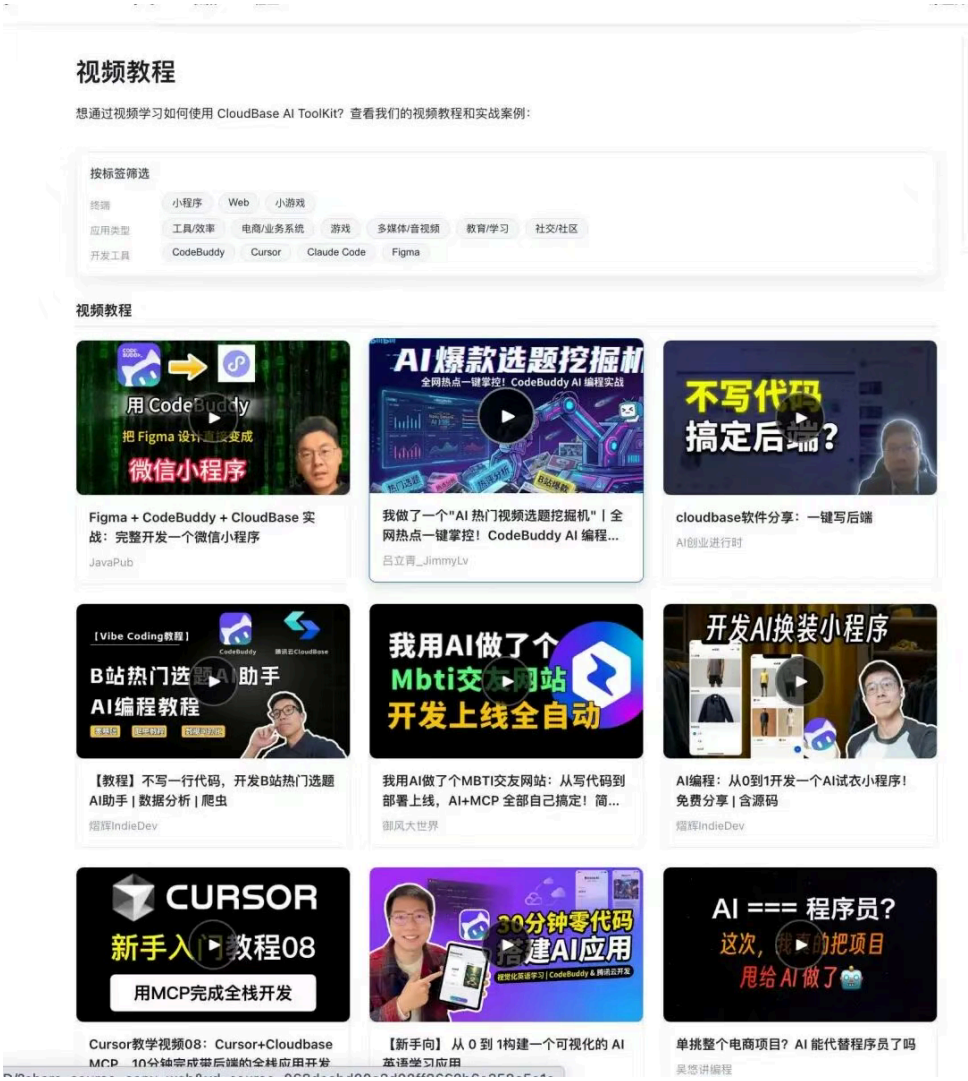


这里有一篇教程教你用免费云开发资源与混元Token，手把手教你用 AI 来开发一个 AI 小程序。



查看完整教程

还可以**查看更多入门视频教程和文章**[9]



终章：Skills 落地踩坑与实战复盘

在做这套 Skills 的过程中，我们最大的感触是：**现在的 AI 编程，其实处于一个“高智商、低纪律”的阶段。** 别看各大工具都号称支持 Skills，真用起来全是坑。

以下是几个我们里跟 AI “斗智斗勇”换来的经验教训：

1. 现状：AI 为什么会“装死”？

明明配了 Skill，AI 还是视而不见。这背后的原因其实涉及大模型的两个底层逻辑：

- **注意力权重（Attention Bias）**：在大模型的 Transformer 架构中，它会根据你的 Prompt 实时计算每个词的权重。当你聊得越深，你提的业务需求权重就会越高，而作为背景板的外部 Skills 权重就会被“稀释”。
- **决策惰性（Inference Laziness）**：调用 Skill 本质上是一次 **Tool Use（工具调用）**。模型在推理时会进行“路径评估”：如果它觉得自己脑子里的预训练数据（通常是 Localhost 模式）就能生成一段“看起来正确”的代码，它就会为了节省推理资源而跳过外部工具调用。
- **激活率落差**：在我们的回归测试中，如果不加干预，AI 的主动调用率只有 **20%** 左右。它宁愿“盲目裸奔”，也不愿意翻书。

2. “驯服” AI 的三套硬核解法

既然 AI 会产生决策惰性，我们就得用工程手段拉高它的“警觉性”。

方案 A：最土但最稳的“首行注入”

如果你的项目不容许犯错，就在提问的最开头带上这句“咒语”：

> **You MUST read the cloudbase-guidelines skill FIRST when working with CloudBase projects.**

- **原理**：利用 **首因效应（Primacy Effect）**。模型对输入序列最前端的信息具有天然的高关注度。通过这种强行注入，能把 Skill 的激活率拉升。

强烈建议大家在试用过程中也带上这句 Prompt，或者按照方案 B 来执行。

方案 B：项目级的“家法” (System Rules)

在项目根目录创建 `CLAUDE.md` 或 `AGENT.md`，增加项目级别的约束规则。这样就不用每次都在 prompt 中 携带上面的咒语，不过这个方法也存在一定的缺陷，相比每次都首行注入，随着对话的上下文越来越长，也会存在 AI 不遵循的情况，接下来就可以看看方案 3 的解法。

```
# CloudBase Workflow
```

```
0. You MUST read the cloudbase-guidelines skill FIRST when working with CloudBase project
1. 评估：写代码前，必须显式评估可用 Skills 列表。
2. 承诺：必须在输出中陈述“我需要调用某个 Skill”的理由。
3. 执行：禁止跳过规矩直接写代码。
```

方案 3：自动化“强制拦截” (Forced Eval Hook)

如果说前两个方案是靠“嘴”叮嘱 AI，那方案 3 就是靠**“物理拦截”**

技术专家 Scott Spence 在他的实测中发现，最有效的方式是利用编辑器的 **Hook（钩子）机制**。

- **它的本质是“中间件”**：这不再是你手动输入的 Prompt。通过配置（如 `.claude/settings.json`），脚本会在你按下回车的那一刻，自动拦截并“魔改”你的问题。
- **强制“表态”才能“干活”**：这个钩子会强制 AI 在输出任何代码前，必须先生成一段 **评估报告**。AI 必须逐一陈述：*“当前有哪些 Skill 可用？针对这个需求，我需不需要调用它们？理由是什么？”*
- **消除“决策惰性”**：AI 有个毛病，只要它觉得脑子里的旧数据能糊弄过去，它就不会翻书。但这种 Hook 强迫 AI 必须白纸黑字写下“我要用这个工具”，一旦它在开头表了态，后面的代码生成就会严格遵守这个承诺。
- **激活率的质变**：根据 Scott 的数据，这种“拦截+评估”的手段能将激活率从 **20% 暴力拉升到 84%**

延伸阅读：**How to Make Claude Code Skills Activate Reliably**[10]

3. 架构复盘：为什么必须是“总纲 + 独立插件”？

在开发过程中，我们发现 AI 最大的毛病是**“端不分”**。它分不清小程序、Web 和 Node.js 的环境边界。我们没有搞一个“全家桶” Skill，而是采用了 **“1 个总纲 (Guidelines) Skill + 21 个独立 Skill”** 的分布式架构。这背后有三个核心逻辑：

A. 解决“语义污染”与环境误判

Web、小程序、Node.js 的 SDK 方法名极其相似（如都有 `init`，`callFunction`）。

- **痛点**：如果全塞进一个 Skill，AI 经常在写小程序逻辑时，顺手掏出 Web 端的语法。这种**语义污染**是造成代码无法上线的元凶。
- **解法**：我们将 Skill 按端进行物理拆分。配合 `cloudbase-guidelines` 这种 **“入口点” (Entry Point)**，先指挥 AI 判定项目环境（如：识别到 `app.json` 即为小程序）。
- **收益**：环境判定后，AI 只会加载相关的子 Skill。这把干扰项直接屏蔽掉，**搜索空间缩小了 90%，推理精度自然大幅提升。**

B. 开发者可以“精准点菜”

- **痛点**：全能包（单体架构）太重。当你想让 AI 专门解决“微信支付”或“安全规则”这种特定问题时，AI 的注意力会被庞大的全量手册稀释。

- **解法**：独立插件支持**局部强化**。你可以直接下令：*“调用 `auth-web` 检查实现手机号登录”*。
- **收益**：这种设计允许开发者直接干预 AI 的决策路径。在 AI 逻辑混乱时，通过唤起特定子 Skill 强行把它的思维拉回到正确的窄道上。

未来规划

我们将持续推出：

- 小程序集成微信支付集成技能
- 小程序集成虚拟支付技能
- 小游戏集成云开发技能
- 云函数部署全栈后端应用技能
- 更多全栈应用开发模板和技能等

欢迎访问

CloudBase Skills GitHub 仓库[11]

CloudBase MCP GitHub 仓库[12]

了解更多详情！

写在最后：从“代码生成”到“生产级交付”的跨越

折腾了这么多 Skills 之后，我们最大的感触是：**AI 编程的生产力，不取决于模型能写出多么精妙的逻辑，而取决于你对它生成的代码具备多少“工程约束力”。**

1. 核心本质：将“隐性经验”转化为“程序性知识”

目前的 Agent 依然处于“高智商、零经验”的状态，它们是博学的“实习生”，却对真实的物理底座缺乏敬畏。**CloudBase Skills 的本质，是给 AI 注入一套工程化的“肌肉记忆”。** 通过 `guidelines` 的语义导航和原子 Skill 的强约束，我们将云开发八年来沉淀的填坑经验（如行权限隔离、多端原生上下文、高并发限流）转化为了 AI 的**前置判定条件**。这种**程序性知识（Procedural Knowledge）**的注入，让 AI 真正理解了“正确且安全”的交付标准。

2. 范式转换：Agent 必须具备“环境感知力”

Agent 进化的下一站，不仅仅是 Reasoning（推理）的增强，更是对 **Infrastructure（基础设施）感知力**的补齐。

- **Skills 守住工程下限**：它是一套“防呆协议”。通过场景化工作流和报错锚点，强制 AI 在编写阶段就化解鉴权漏洞和多端环境差异，彻底终结“代码只活在 `localhost` ”的幻觉。

- **CloudBase 承载逻辑上限**：作为全栈底座，它让 AI 的逻辑输出不再是网页里的文本，而是能直接部署、调通、并产生真实业务价值的生产级资源。

3. 确定性是 AI 开发的唯一度量衡

我们不应该期待 AI 自动变得“完美”，而应该通过工程手段让它变得“稳定”。**AI 提供逻辑的上限，Skills 守住工程的下限，而 CloudBase 则是承载这一切的物理底座。**

我们正处于从“氛围感编程 (Vibe Coding)”向“生产级交付”跨越的节点。如果你也厌倦了在对话框里修 Bug，欢迎安装 CloudBase Skills。给你的 AI 立个规矩，让它真正从一个“聊天搭子”进化为能帮你上线产品的“资深合伙人”。

如有问题或建议，欢迎在评论区交流。

名词解释

- **Vibe Coding**：一种依赖 AI 直觉、快速生成原型但缺乏工程严谨性的编程方式。
- **Procedural Knowledge**：程序性知识。指关于“如何操作”的知识，让 AI 掌握特定场景下的标准作业流程（SOP）。
- **MCP**：模型上下文协议。AI 的**“连接器”**，让其能直接操作数据库、文件系统或云资源。
- **Agent Skills**：能力插件包。AI 的**“专家手册”**，注入特定领域的最佳实践和工程规范。
- **横向越权**：安全漏洞。指攻击者通过篡改 ID 等参数，非法访问到同级别其他用户的数据。
- **安全规则**：底座级权限控制。在数据库入口处强制执行访问校验，是防御越权的核心手段。

立即开始

在你的终端输入以下命令，为你的 AI 助手注入“生产级直觉”：

```
npx skills add tencentcloudbase/skills
```

参考资料

[1] Agent Skill : *https://agentskills.io/*

[2] react-best-practices : *https://vercel.com/blog/introducing-react-best-practices*

[3] 视频制作 Skill: *https://x.com/Remotion/status/2013626968386765291*

[4] Vercel 推出了 skills 命令: *https://skills.sh/*

[5] 云开发 CloudBase: *https://tcb.cloud.tencent.com/*

[6] CloudBase Skills: *https://github.com/TencentCloudBase/Skills*

[7] CloudBase MCP: *https://github.com/TencentCloudBase/CloudBase-MCP*

[8] 查看具体使用指引: *https://docs.cloudbase.net/ai/cloudbase-ai-toolkit/*

[9] 查看更多入门视频教程和文章: *https://docs.cloudbase.net/ai/cloudbase-ai-toolkit/tutorials*

[10] How to Make Claude Code Skills Activate Reliably: *https://scottspence.com/posts/how-to-make-claude-code-skills-activate-reliably*

[11] CloudBase Skills GitHub 仓库: *https://github.com/TencentCloudBase/Skills*

[12] CloudBase MCP GitHub 仓库: *https://github.com/TencentCloudBase/CloudBase-MCP*



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容

公众号



阅读 1.3万

留言

写留言

还没有留言，来写首评